The Branchless Calculus: Algebra, Geometry, and Discrete Calculus of Constant-Time Computation over Machine Words

Blackcat Informatics Research

Grounded in the PurRDF `purrdf-branchless` kernel corpus
(bcinr provenance pin `1e92b213`, v26.10.26 wave)

2026

Abstract

This thesis develops the mathematical structure underlying branchless computation: the replacement of data-dependent control flow with arithmetic that is *correct by construction* over an entire input domain at once. Three lenses are developed. *Algebra*: comparison and selection form a two-element idempotent semiring action on machine words; we give the exact identities of the isolate/clear/mask trio ($\mathsf{blsi}, \mathsf{blsr}, \mathsf{blsmsk}$) and prove the correctness of mask-form compare-exchange. *Geometry*: Morton and Hilbert space-filling curves are treated as bit-interleaving isometries between $\mathbb{N}^d$ grids and $\mathbb{Z}/(2^{dn})$; we prove the round-trip involution of the Hilbert transposition operator and the level-monotonicity of the fixed-step encodings. *Calculus*: prefix scans are the discrete antiderivative Δ^{-1}; we show Hillis–Steele doubling is a parallel-in-time quadrature, prove segmented scans compute integrals against indicator measures, and derive Gray-code successor as a discrete derivative identity. Every result is paired with the differential-oracle discipline used to admit it in practice: a claim about a kernel is a statement of bit-exact equality against an independently structured reference, and a test that cannot fail is not evidence.

Contents

1 Introduction **2**

2 Algebra of Masks and Selection **3**
2.1 The mask semiring . . . 3
2.2 The isolate–clear–mask trio . . . 3
2.3 Funnel shifts as word-level quotients . . . 4

3 Geometry: Space-Filling Keys **5**
3.1 Morton codes as bit interleaving . . . 5
3.2 The Hilbert curve as a reflective itinerary . . . 5

4 Discrete Calculus of Scans **7**
4.1 The scan as antiderivative . . . 7
4.2 Segmented scans as integration against indicators . . . 7
4.3 Gray codes: a discrete derivative identity . . . 8
4.4 Carry-save summation . . . 8

5 Verification as Equational Reasoning **9**
5.1 Differential oracles . . . 9
5.2 The round trip as the canonical invariant . . . 9
5.3 Two worked escape cases . . . 9

6 Conclusion **11**

Chapter 1

Introduction

A branchless program makes no decision. Where a conventional implementation asks *which* of two paths the data demands, a branchless implementation computes *both* and arithmetically discards one. The price is a constant factor in work; the payoff is that the execution time becomes a function of the *type* of the input (its width, its lane count) and not of its *value*. Formally, for a kernel f and inputs x, x' of identical shape,

$$T(f(x)) \;=\; T(f(x')), \tag{1.1}$$

i.e. latency is value-independent. This is the Radon Law of the source corpus: every execution path is deterministic, memory-safe, and constant-time.

Constant time is usually argued for cryptography. This thesis argues it for *correctness engineering*: when control flow is arithmetic, the proof obligations become equational, and equations can be checked mechanically by differential testing against independent oracles. The PurRDF `purrdf-branchless` corpus serves as the running subject: 30+ kernels, each admitted by a Chicago-style court — an assertion of the kernel against a *differently formulated* reference implementation, over exhaustive small domains, boundary words, and deterministic pseudorandom sweeps.

Notation. Words live in $\{0,1\}^{64}$, identified with $[0, 2^{64})$. Bit 0 is least significant. $\mathbf{1} \equiv 2^{64} - 1$. The Iverson bracket $[\,P\,]$ is 1 if P holds and 0 otherwise. $\mathsf{w}[i]$ denotes bit i of word w. All arithmetic on words is modulo 2^{64} unless stated.

Chapter 2

Algebra of Masks and Selection

2.1 The mask semiring

Definition 2.1 (Mask algebra)**.** *For $c, a, b \in \{0,1\}^{64}$, define*

$$\mathcal{M}(c, a, b) \;=\; (c \,\&\, a) \mid (\neg c \,\&\, b), \tag{2.1}$$

where $\neg$ denotes bitwise complement. We call c the mask *(all-ones = true), a the taken value, b the fallen value.*

Proposition 2.2. *$\mathcal{M}(c, a, b) = (a \,\&\, c) \mid (b \,\&\, \neg c)$ satisfies $\mathcal{M}(\mathbf{1}, a, b) = a$, $\mathcal{M}(0, a, b) = b$, and $\mathcal{M}(c, x, x) = x$ for all c.*

Proof. Per bit: the map $\{0,1\}^3 \to \{0,1\}$, $(c_i, a_i, b_i) \mapsto (a_i \wedge c_i) \vee (b_i \wedge \neg c_i)$ is the multiplexer of Boolean algebra; substituting $c_i = 1$, $c_i = 0$, $a_i = b_i$ gives the three claims. Bitwise operations act pointwise, so the word-level statements follow lane-wise. □

Remark 2.3. *The 'xor' form $\mathcal{M}(c, a, b) = b \oplus ((a \oplus b) \,\&\, c)$ is algebraically identical (the Boolean ring is commutative under $\oplus$, and $\&$ is its meet) and is the form hardware compiles to a single conditional move: the compare-exchange kernel*

$$\mathsf{cswap}(a, b) \;=\; \big(m,\; a \oplus d,\; b \oplus d\big), \qquad m = [\,a > b\,], \quad d = (a \oplus b) \,\&\, (0 \ominus m), \tag{2.2}$$

where $0 \ominus m \equiv (0 - m) \bmod 2^{64}$ is the arithmetic negation of the mask, exchanges a and b exactly when $a > b$, and executes the identical instruction stream for every input pair. The arithmetic negation acts as a sign-extension operator*: for $p \in \{0,1\}$, $0 \ominus p = -p \bmod 2^{64}$ equals 0 or $\mathbf{1}$.*

2.2 The isolate–clear–mask trio

Three low-bit operators anchor the bitset algebra of the corpus. Define, for $x \in \{0,1\}^{64}$ with $x \neq 0$ and $t = \mathrm{tz}(x)$ the index of its lowest set bit:

$$\mathsf{iso}(x) = x \,\&\, (-x) \qquad = 2^t, \tag{2.3}$$

$$\mathsf{clr}(x) = x \,\&\, (x - 1) \qquad = x - 2^t, \tag{2.4}$$

$$\mathsf{msk}(x) = x \oplus (x - 1) \qquad = 2^{t+1} - 1. \tag{2.5}$$

Theorem 2.4 (Trio identities)**.** *For all $x \neq 0$ with lowest set bit t:*

1. $\mathsf{iso}(x) \mid \mathsf{clr}(x) = x$ *and* $\mathsf{iso}(x) \,\&\, \mathsf{clr}(x) = 0$*;*

2. $\mathsf{msk}(x) = \mathsf{iso}(x) \mid (\mathsf{iso}(x) - 1)$*;*

3. clr *iterated* k *times* ($0 \le k \le \mathrm{pop}(x)$) *removes the* k *lowest set bits; hence* $\mathrm{pop}(\mathsf{clr}^{(k)}(x)) = \mathrm{pop}(x) - k$*;*

4. $\mathsf{msk}(0) = \mathbf{1}$ *(the unique degenerate case: the difference* $x - 1$ *wraps).*

Proof. (1) Writing $x = h \cdot 2^{t+1} + 2^t$ with h's bits strictly above t: $-x = \neg x + 1$ flips all bits above t and leaves bits $0..t$ as $0..0\,1$, so $x \,\&\, -x = 2^t$ and $x \,\&\, (x-1)$ clears exactly bit t. Disjointness gives both identities. (2) $2^t | (2^t - 1) = 2^{t+1} - 1$ since the operands are disjoint and the second is all-ones below t. (3) Each application strictly decreases pop by one (part 1), and $x - 2^t = \mathsf{clr}(x)$ removes exactly the lowest set bit; induction over k. (4) $0 \oplus (0 - 1) = 0 \oplus \mathbf{1} = \mathbf{1}$. □

Identity (3) is the engine of the collision peel used in the corpus's hash-staged lane search: iterating *hits* &= *hits* − 1 walks the set bits of *hits* in ascending order, so a hash collision costs one scan step per colliding element and never binds a key to a foreign ordinal.

2.3 Funnel shifts as word-level quotients

View the pair $(h, l) \in \{0,1\}^{128}$ as the integer $H = 2^{64}h + l$. The funnel shift right by s bits is

$$\mathsf{funnel}_r(h, l, s) = \left\lfloor H \,/\, 2^s \right\rfloor \bmod 2^{64} = (h \gg s) \mid (l \ll (64 - s)), \tag{2.6}$$

with the classical degeneracy at $s = 0$: the term $l \ll 64$ is a shift by the word width, undefined on most hardware. The corpus formulation resolves it with the *zero-mask* $z = 0 \ominus [\, s \neq 0 \,]$:

$$\mathsf{funnel}_r(h, l, s) = (l \gg (s \& 63)) \;\mid\; \big(h \ll ((64 - s) \& 63) \;\&\; z\big), \qquad \mathsf{funnel}_r(h, l, 0) = l. \tag{2.7}$$

Proposition 2.5. *For* $s' = s \bmod 64$*:* funnel_r *agrees with* $\lfloor H / 2^{s'} \rfloor \bmod 2^{64}$ *for* $s' \neq 0$*, equals* l *at* $s' = 0$*, and the same construction with* $\gg, \ll$ *exchanged gives* $\mathsf{funnel}_l(h, l, 0) = h$ *and the left quotient* $H \cdot 2^{s'} \bmod 2^{64}$ *truncated otherwise. In particular both agree with a* 128*-bit reference for all* $s \in \{0, \ldots, 66\}$*, so the masking behaviour at* $s \in \{64, 65, 66\}$ *is exactly the* $s \bmod 64$ *behaviour.*

Proof. For $s' \neq 0$, both component shifts are in $[1, 63]$, hence defined, and the two extracted halves are disjoint and cover exactly the window bits $[s', s' + 64)$ of H. At $s' = 0$ the mask $z = 0$ annihilates the cross term, leaving the claimed identity. The 128-bit reference $((H \gg s')) \bmod 2^{64}$ matches lane-wise. □

Chapter 3

Geometry: Space-Filling Keys

3.1 Morton codes as bit interleaving

Definition 3.1 (Morton / Z-order code)**.** *The d-dimensional Morton code of level n is the bit interleaving*

$$\mathsf{morton}_d(x_0,\dots,x_{d-1}) \;=\; \sum_{j=0}^{d-1} 2^j\,\mathsf{spread}_d(x_j)\,, \qquad \mathsf{spread}_d(x) = \sum_{i=0}^{n-1} x_i\,2^{di}, \tag{3.1}$$

where x_i is bit i of x_j. For $d = 2$ the spread is the composition of five parallel bit-transposition stages with masks $0xFFFF\,0000FFFF$, $0x00FF00FF00FF00FF$, $0x0F0F\ldots$, $0x3333\ldots$, $0x5555\ldots$ — the classical divide-and-conquer transposition doubling the stride each stage.

Theorem 3.2 (Round trip)**.** $\mathsf{compact}_d(\mathsf{morton}_d(x_0,\dots,x_{d-1})) = (x_0,\dots,x_{d-1})$ *for all coordinates with $x_j < 2^n$, where* $\mathsf{compact}_d$ *selects the congruent residue classes $j \bmod d$ of bit positions and packs them right.*

Proof. Interleaving places bit i of x_j at position $di + j$; the classes $\{\, di + j : i \,\}$ are pairwise disjoint and their union is $\{0,\dots,dn-1\}$, so selection inverts interleaving exactly. □

The corpus implements the $d = 2$ spread/compact by *composition* over the deposit/extract kernels $\mathsf{spread}_2(x) = \mathsf{deposit}(x, \texttt{0x5555}\ldots)$, $\mathsf{compact}_2(m) = \mathsf{extract}(m, \texttt{0x5555}\ldots)$ — one home for the transposition algebra, no second body. The $d = 3$ masks are `0x1249 2492 4924 9249` and its two rotations, selecting the residue classes $0, 1, 2 \bmod 3$ over 21 bits per axis.

3.2 The Hilbert curve as a reflective itinerary

Where Morton preserves locality poorly across the code space, Hilbert's curve visits every cell with unit steps. Its geometry is governed by a single involution.

Definition 3.3 (Transposition operator)**.** *Let R be reflection about the anti-diagonal (the coordinate swap) and $C_r, C_{\bar r}$ the complementations of x and y by the half-width $\frac{n}{2}$. Define*

$$T(x,y) \;=\; \big(C_{\bar r}(x),\, C_r(y)\big) \circ R, \tag{3.2}$$

i.e. swap, then complement x when the reflected bit $\bar r$ is set and complement y when r is set.

Theorem 3.4 (T is an involution)**.** $T \circ T = \mathrm{id}$.

Proof. Coordinate swap R is an involution, as is each complementation C ($C \circ C = \mathrm{id}$, complements commute). $T = (C_{\bar{r}} \times C_r) \circ R$ is a composition of commuting involutions' product with an involution, hence an involution: $T \circ T = (C_{\bar{r}} \times C_r) \circ R \circ (C_{\bar{r}} \times C_r) \circ R = (C_{\bar{r}}^2 \times C_r^2) \circ R^2 = \mathrm{id}$, using that the product complementations commute with R (they act on different coordinates after the swap is accounted for by the paired indices $r, \bar{r}$). □

Theorem 3.5 (Level round trip)**.** *Let* $\mathsf{hilb}_\ell : \{0, \ldots, 2^\ell - 1\}^2 \to \{0, \ldots, 4^\ell - 1\}$ *be the level-*ℓ *Hilbert encoding obtained by descending the quadrant recursion with operator* T*, consuming the distance digits MSB-first, and let* hilb_ℓ^{-1} *be the decode applying the same recursion to coordinates. Then*

$$\mathsf{hilb}_\ell^{-1}(\mathsf{hilb}_\ell(x, y)) = (x, y) \quad \textit{and} \quad \mathsf{hilb}_\ell\big(\mathsf{hilb}_\ell^{-1}(d)\big) = d \tag{3.3}$$

for all cells and distances, and the induced path is contiguous*: consecutive distances index edge-adjacent cells.*

Proof. By Theorem 3.4 the quadrant transformation applied in encode is undone exactly by the decode's application of T, and the digit schedules are mutual inverses (encode peels the coordinator bit pair (r_x, r_y) at stride $s = 2^{\ell-1-i}$ while decode prepends them at the same stride), so a straightforward induction on ℓ gives both round trips. Contiguity is the defining quadrant concatenation property of Hilbert's curve: within a quadrant the recursion is contiguous by hypothesis, and the operator T rotates each quadrant's entry/exit cells so the four sub-paths chain end-to-end. □

Remark 3.6 (Level parameterisation and the fixed-step form)**.** *The corpus implementation fixes the loop shape at* 32 *steps (stride* $s = 2^{30}$ *descending to* 1*) and clamps* $\ell \in [0, 30]$*, so a single branchless body serves every level. Steps past the active level are annihilated by an* s*-presence mask* $g = 0 \ominus [\, s \neq 0 \,]$ *multiplying the swap flag — dead iterations are complete no-ops, preserving value-independence of the execution path. The level-1 orientation table,*

$$(0, 0) \mapsto 0, \quad (0, 1) \mapsto 1, \quad (1, 1) \mapsto 2, \quad (1, 0) \mapsto 3, \tag{3.4}$$

pins the packed-MSB-first convention; the geo-kernel consumer (`hilbert_path`) delegates to this single home, and the conformance vectors assert byte-identical cell identifiers before and after the delegation — any change is an over-refusal-class regression.

Chapter 4

Discrete Calculus of Scans

4.1 The scan as antiderivative

Definition 4.1 (Difference and sum operators)**.** *On sequences $a : \{0, \dots, n-1\} \to \mathbb{Z}/2^{32}$ define the forward difference $\Delta a_i = a_{i+1} - a_i$ and the (inclusive) prefix sum*

$$(\Sigma a)_i = \sum_{k \le i} a_k, \qquad (\Sigma^- a)_i = \sum_{k < i} a_k. \tag{4.1}$$

Σ is the discrete antiderivative: $\Delta\Sigma a = a$ up to boundary, and Σ^- is the calibrated inverse with $(\Sigma^- a)_0 = 0$.

The corpus's `exclusive_scan_u32x16` is exactly Σ^-: its defining invariant, pinned by test, is $out_0 = 0$; its sibling `prefix_sum_u32x16` is Σ.

Theorem 4.2 (Hillis–Steele doubling)**.** *Define $a^{(0)} = a$ and $a_i^{(k+1)} = a_i^{(k)} + a_{i-2^k}^{(k)}$ (absent terms 0). Then $a^{(\lceil \log_2 n \rceil)} = \Sigma a$.*

Proof. Induction: $a_i^{(k)} = \sum_{j=\max(0,i-2^k+1)}^{i} a_j$. For $k = 0$ this is a_i. The step merges two windows whose lengths sum to 2^{k+1} and whose union is the required range. At $2^k \ge n$ the window covers $[0, i]$. □

This is a quadrature in the group $(\mathbb{Z}/2^{32}, +)$: the scan computes n "integrals" in $\lceil \log_2 n \rceil$ vectorized passes, work $O(n \log n)$ but *depth* $O(\log n)$ — the parallel-in-time calculus that SWAR packs into register lanes.

4.2 Segmented scans as integration against indicators

Definition 4.3 (Segment indicator)**.** *For segment flags $f \in \{0,1\}^n$ (1 = segment start), define the segment identity of position i: $\sigma(i) = \max\{\, j \le i : f_j = 1 \,\}$ (or -1 if none).*

Theorem 4.4 (Segmented scan = measure integral)**.** *Let $S = \{\, (j, i] : 0 \le j < i < n,\ \sigma(i) = j \,\}$ be the partition induced by f. The segmented prefix sum*

$$\mathsf{segscan}(a, f)_i \;=\; \sum_{k=\sigma(i)}^{i} a_k \;=\; \int_{[\,\sigma(i),\, i\,]} a \, d\mu_f, \qquad \mu_f = \sum_i [\,\sigma(i) \ge 0\,]\ \delta_i, \tag{4.2}$$

is computed branchlessly by the reset-mask recurrence

$$r_i \;=\; a_i \;+\; (1 - f_i)\, r_{i-1}, \qquad r_{-1} = 0, \tag{4.3}$$

whose mask form in the corpus is acc &= ¬(0 ⊖ f_i); acc += a_i.

Proof. Unrolling the recurrence gives $r_i = \sum_{k=\sigma(i)}^{i} a_k \prod_{m=\sigma(i)+1}^{i}(1-f_m) + \sum_{m>\sigma(i),\, m\le i} a_m \prod_{m'>m}^{i}(1-f_{m'})$; every product containing a flag $f_m = 1$ with $m > \sigma(i)$ vanishes, which cannot happen by maximality of $\sigma(i)$; the surviving sum is exactly $\sum_{k=\sigma(i)}^{i} a_k$. The pinned corpus example $a = (1,\dots,8)$, $f = (0,0,0,1,0,0,1,0)$ evaluates to $(1,3,6,4,9,15,7,15)$: check $\sigma(3) = 3 \Rightarrow r_3 = 4$, $\sigma(7) = 6 \Rightarrow r_7 = 7 + 8$. □

4.3 Gray codes: a discrete derivative identity

Definition 4.5 (Gray code)**.** $\mathsf{gray}(n) = n \oplus (n \gg 1)$.

Theorem 4.6 (Unit successive difference)**.** *For all n:* $\mathsf{gray}(n) \oplus \mathsf{gray}(n+1) = 2^{\mathrm{tz}(n+1)}$*, and* gray *is a bijection $\{0,1\}^{64} \to \{0,1\}^{64}$ with inverse $n \mapsto n \oplus (n \gg 1) \oplus (n \gg 2) \oplus \cdots$ (the prefix xor $\Sigma_\oplus$).*

Proof. n and $n+1$ agree above bit $t = \mathrm{tz}(n+1)$ and differ exactly in bits $0..t$ (the carry chain). In $n \oplus (n \gg 1)$ the differing contributions above bit t cancel pairwise; below and at t exactly bit t survives. Bijectivity: the map $n \mapsto n \oplus (n \gg 1)$ is lower-triangular over $\mathbb{F}_2^{64}$ with unit diagonal, hence invertible; its inverse is the cumulative xor (the xor-analogue of Σ), a fact the corpus tests via the Gray-code spot identity $\mathsf{prefix_xor}([0,1,\dots,7]) = (0,1,3,0,4,1,7,0)$. □

Thus Gray enumeration is a Hamiltonian path on the hypercube Q_{64}, and its step direction is the 2-adic valuation of $n+1$ — a discrete derivative whose magnitude classifies each step.

4.4 Carry-save summation

The horizontal byte sum of a word $s = \sum_{i=0}^{7} \mathsf{w}[i]$, $0 \le s \le 2040$, is computed by the carry-save identity: with $v = \sum_i \mathsf{w}[i]\, 2^{8i}$,

$$v \;=\; \big(v \,\&\, \texttt{0x00FF00FF}\ldots\big) \;+\; \big(v \gg 8 \,\&\, \texttt{0x00FF00FF}\ldots\big) \tag{4.4}$$

is exact — no lane overflows because $0xFF + 0xFF \le 0x1FF < 2^9$ and the stride-8 lanes leave a guard bit per stage. Five stages collapse eight lanes to one; the corpus bound $\texttt{horizontal_sum} \le 2040 = 8 \times 255$ is the measure of that exhaustion, and its pinned canonical value on $u64$: : $from_le_bytes([3,1,4,1,5,9,2,6])$ is 31.

Chapter 5

Verification as Equational Reasoning

5.1 Differential oracles

Every algebraic claim above is admitted operationally by the same scheme. For kernel k and independent reference k^* — one *structurally different* formulation (scalar bit loops vs. SWAR; branchy textbook recursion vs. mask arithmetic; `sorted()` vs. comparator networks) — the admission court asserts

$$\forall x \in B \cup F_n : \quad k(x) = k^*(x), \tag{5.1}$$

where B is a fixed boundary corpus (for $u64$: $\{0, 1, 2^{63}, 2^{64}-1, \texttt{0x8000_0000_0000_0001}, \texttt{0x0F0F_F0F0} \ldots\}$) and F_n is a deterministic xorshift64* stream ($x \mapsto x \oplus (x \ll 13) \oplus (x \gg 7) \oplus (x \ll 17)$, then multiplied by $0x2545F4914F6CDD1D$). Boundary terms kill *range* mutants (wrap, sign); random terms kill *algebra* mutants (associativity swaps, lost masks).

Proposition 5.1 (Mutant sufficiency of the boundary set)**.** *For the isolate–clear–mask trio, $B \cup \{x \le 0xFFFF\}$ exhausts the small-domain behaviour:* $\mathsf{iso}, \mathsf{clr}, \mathsf{msk}$ *act on the lowest set bit only, and $x \le 2^{16}$ realises every (t, h) pair class with $h < 2^{15}$, so any mutant that is not value-identical on the lowest-set-bit decomposition diverges inside the corpus.*

5.2 The round trip as the canonical invariant

Round-trip identities — $\mathsf{compact} \circ \mathsf{spread}$, $\mathsf{decode} \circ \mathsf{encode}$, $\mathsf{extract} \circ \mathsf{deposit}$ — are the geometry chapter's theorems converted into executable form. Their evidentiary power comes from direction asymmetry: an implementation pair can be *mutually consistent* yet jointly wrong (both invert the same mistaken convention), so each round trip is paired with an *orientation pin* — literal values fixed to the external standard. The corpus pins, e.g., $\mathsf{hilb}_1(1, 0) = 3$, $\mathsf{hilb}_1(0, 1) = 1$, and the Eytzinger layout of $\{1..6\}$ as $(0, 4, 2, 6, 1, 3, 5)$, so a shared convention drift cannot pass through the court.

5.3 Two worked escape cases

The 26.10.26 wave recorded two instructive near-misses, both caught statically — by hand-derivation, not by execution:

1. **The shift-by-64 hole.** A proposed two-word field extract used $(lo \gg o) \mid (hi \ll (64 - o))$; at $o = 0$ this is a shift by the word width. The landed form moves the window to $\mathbb{Z}/2^{128}$:

$$W = 2^{64}h + l, \qquad \mathsf{field}_k(W, o) = \left\lfloor W/2^o \right\rfloor \,\&\, \left(2^k - 1\right), \qquad k \in [1, 64], \tag{5.2}$$

   with the width mask expressed $2^{64} - 2^{64-k}$ so that $k = 64$ is the *unclamped* full window — the degenerate case is absorbed by the algebra rather than special-cased in control flow.

2. **Perfect-tree masking.** An off-by-one in a subtree-size table is invisible to search when $n = 2^d - 1$ (perfect tree): the deficit propagates along a path the search never samples. The masking predicate $n \not\equiv 2^d - 1$ therefore belongs in the theorem, not the test suite — and the boundary test must include a non-perfect shape ($n = 6$) whose wrong table *provably* diverges ($\mathsf{lower_bound}(7)$: 5 vs. 6).

Both illustrate the thesis's central discipline: *a green suite is a conjunction of sampled implications; only the algebra states the universal.*

Chapter 6

Conclusion

Branchless computation is not a performance trick; it is a proof technique. When selection is $\mathcal{M}(c, a, b)$, when spatial keys are interleavings and involutions, when prefix aggregation is Δ^{-1}, the gap between the specification and the machine word narrows to an equation — and equations admit mechanical, differential witnesses. The kernel corpus developed here carries each theorem in three simultaneous encodings: the mask arithmetic (executable), the independent oracle (adjudicating), and the literal orientation pins (anchoring convention). The result is code whose correctness argument is not a narrative about testing but a series of small, named identities — $\mathsf{iso}|\mathsf{clr} = x$, $T^2 = \mathrm{id}$, $\Delta\Sigma = \mathrm{id}$ — each of which a machine, or a reviewer, can check in one line.

Bibliography

[1] H. S. Warren, *Hacker's Delight*, 2nd ed., Addison-Wesley, 2012 (bit tricks: blsi/blsr/blsmsk, Gray, funnel).

[2] D. Hilbert, Über die stetige Abbildung einer Linie auf ein Flächenstück, *Math. Ann.* 38 (1891) 459–460.

[3] G. M. B. Morton, *A computer oriented geodetic data base and a new technique in file sequencing*, IBM, 1966.

[4] W. D. Hillis, G. L. Steele, Data parallel algorithms, *Comm. ACM* 29(12) (1986) 1170–1183.

[5] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. (sorting networks, Fig. 51).

[6] P. Eytzinger, *Algebraische Berechnung der Funktion des Logarithmus*, 1603 (breadth-first search layout ancestry).